

1 P2 C5 — An improved lower bound for uphill paths in Q9

We prove $U(Q_9) \geq 2369$. The argument excludes every hypothetical labelling of Q_9 with at most 2368 uphill paths using exact path-count identities, an elementary code bound, Harper's vertex-isoperimetric theorem, and a fully specified finite arithmetic certificate. The known 2400-path construction supplies an upper bound but is not needed here.

1.1 Definitions and exact budget

The vertices of Q_9 are the nine-bit words. A labelling gives them distinct labels and orients every edge from its lower-labelled to its higher-labelled endpoint. An uphill path starts at a *valley* (a vertex with no incoming edge), follows oriented edges, and may consist of its valley alone. Let $p(v)$ be the number of such paths ending at v , and let $P = \sum_v p(v)$. In increasing label order,

$$p(v) = \begin{cases} 1, & v \text{ is a valley,} \\ \sum_{u \rightarrow v} p(u), & \text{otherwise.} \end{cases}$$

Set $A = \{v : p(v) = 1\}$, $B = V(Q_9) \setminus A$, $b = |B|$, s = the number of valleys, and $e_B = |E(Q_9[B])|$. An edge from B to A is impossible: its head would receive at least two paths. Every non-valley of A therefore has exactly one incoming edge, from A . The induced graph $Q_9[A]$ is a forest: on an undirected cycle, its highest-labelled vertex would have two incoming edges and hence $p \geq 2$. Every component of this forest contains exactly one valley, so it has $|A| - s$ edges.

There are 2304 edges in Q_9 . The number of edges incident to B is $9b - e_B$; consequently

$$2304 = (512 - b - s) + (9b - e_B), \quad s + e_B = 8b - 1792. \quad (1)$$

Each uphill path is either a singleton valley or has a last edge. Let C be the vertices of B with an outgoing edge inside B , let $q = |C|$, let $H = B \setminus C$, and write $t_u = \text{outdeg}_B(u)$. The set H is independent: an edge between two of its vertices would have a tail. Every B -sink is also a sink of the full orientation because no $B \rightarrow A$ edge exists. Since $p = 1$ on A , path counting by last edge gives two useful exact forms:

$$\begin{aligned} P &= 2304 + s + \sum_{u \in C} (p(u) - 1)t_u, \\ &= 512 + 8b + E, \quad E = \sum_{u \rightarrow v \in E(Q_9[B])} (p(u) - 2) \geq 0. \end{aligned} \quad (2)$$

Suppose for contradiction that $P \leq 2368$. Equation (1) and $s \geq 1$ give $b \geq 225$; equation (2) gives $b \leq 232$. Put $\delta = 232 - b \in \{0, \dots, 7\}$. Then

$$E \leq 8\delta, \quad W := \sum_{u \in C} (p(u) - 1)t_u = P - 2304 - s \leq 63. \quad (3)$$

For a core vertex $u \in C$, write a_u for its neighbours in A and i_u for its incoming B -degree. Then $a_u + i_u + t_u = 9$ and $p(u) \geq a_u + 2i_u = 9 - t_u + i_u$. In particular $1 \leq t_u \leq 8$, and its cost is at least

$$(p(u) - 1)t_u \geq t_u \max\{1, 8 - t_u\} \geq 7. \quad (4)$$

Thus $q \leq 9$. For $t = 1, \dots, 8$, the lower costs are respectively 7, 12, 15, 16, 15, 12, 7, 8. This is a necessary relaxation; it does not assert that every locally allowed type is realizable.

1.2 The length-nine code bound $A(9, 4) \leq 20$

We use a self-contained bound. Let $D \subseteq \{0, 1\}^9$ be an even-parity code with minimum Hamming distance at least four, and write $M = |D|$. For $j = 4, 6, 8$, let A_j be the average number of codewords at distance j from a codeword. Then $M = 1 + A_4 + A_6 + A_8$. The squared sums of the nine coordinate characters and of their 36 pairwise products yield, respectively,

$$9 + A_4 - 3A_6 - 7A_8 \geq 0, \quad 36 - 4A_4 + 20A_8 \geq 0. \quad (5)$$

Indeed, for $s_i = \sum_{x \in D} (-1)^{x_i}$, expansion of $\sum_i s_i^2/M$ gives the first expression. For $t_{ij} = \sum_{x \in D} (-1)^{x_i + x_j}$, expansion of $\sum_{i < j} t_{ij}^2/M$ gives the second; a pair at distance d contributes $((9 - 2d)^2 - 9)/2$. Two codewords each at distance eight from a fixed word would be at distance two from each other, so $A_8 \leq 1$. Elimination from (5) gives

$$M \leq 16 + \frac{16}{3}A_8 \leq \frac{64}{3} < 22.$$

Suppose $M = 21$, and let $N_j = MA_j$, an even integer counting ordered pairs. The last inequality forces $A_8 \geq 15/16$, while $N_8 \leq 21$, hence $N_8 = 20$. Since $N_4 + N_6 = 400$, the nonnegative squared-character sums become $449 - 4N_6$ and $4N_6 - 444$. Thus $111 \leq N_6 \leq 112$; its evenness forces $N_6 = 112$ and $\sum_i s_i^2 = 1$. But each s_i is a sum of 21 signs and is odd, so $\sum_i s_i^2 \geq 9$. Contradiction. Therefore $M \leq 20$.

The same bound holds without the even-parity assumption: puncture one coordinate of an arbitrary distance-four length-nine code, then append the parity bit of each resulting eight-bit word. The replacement code is even, has the same size, and retains minimum distance at least four.

1.3 A local forest constraint

Let $x \in B$, and suppose a of its nine neighbours lie in A . Each pair of these a neighbours has a unique second common cube neighbour $z \neq x$, at distance two from x . If $z \in A$, regard it as an edge between that pair. These edges form a forest on the a neighbours: a cycle in the pair graph lifts to a subdivided cycle in $Q_9[A]$. Hence at most $a - 1$ second common neighbours lie in A , and at least

$$\binom{a}{2} - (a - 1) = \binom{a - 1}{2} \quad (6)$$

lie in B , all with the same parity as x . For $a = 0, 1$, read the lower bound as zero. This lemma applies to sinks and cores alike.

1.4 Harper's neighbourhood bound and the minority-sink range

We use the following classical form of Harper's vertex-isoperimetric theorem: among all c -element subsets of Q_8 , the closed vertex neighbourhood has minimum size for an initial segment in simplicial order: increasing Hamming weight, then lexicographic order of the increasing coordinate-support tuples. We cite L. H. Harper, "Optimal numberings and isoperimetric problems on

graphs,” *Journal of Combinatorial Theory* **1** (1966), 385–393, for this minimization theorem. Identify either parity class of Q_9 with Q_8 by deleting one bit and restoring it from parity. The open Q_9 neighbourhood of a set in that class corresponds to the closed Q_8 neighbourhood. Let $N_8(c)$ denote the resulting minimum.

Suppose H meets both parities, and choose the minority side H_m of size $c \geq 1$. Since H is independent, its opposite-parity neighbours avoid the majority sinks. Since $|H| = b - q$,

$$N_8(c) - c \leq 256 - b + q = 24 + \delta + q \leq 40. \quad (7)$$

Also $c \leq |H|/2 \leq 116$. Direct evaluation of the simplicial initial segments gives $N_8(12) - 12 = 40$ and $\min_{13 \leq c \leq 116} (N_8(c) - c) = 42$; hence $c \leq 12$. The following complete 256-step standard-library replay evaluates the finite table. Harper’s theorem, rather than this code, supplies the minimization assertion.

```
from itertools import combinations
covered = set()
N = [0]
for k in range(9):
    for coords in combinations(range(8), k):
        v = sum(1 << j for j in coords)
        covered.add(v)
        covered.update(v ^ (1 << j) for j in range(8))
    N.append(len(covered))
assert len(N) == 257
assert N[11] - 11 == 37
assert N[12] - 12 == 40
assert min(N[c] - c for c in range(13, 117)) == 42
```

For a minority sink x , let d_x be its neighbours among the q_M majority-parity cores, and let q_m be the minority-parity core count. Its other $9 - d_x$ neighbours lie in A . Applying (6) and counting all other minority-parity B vertices gives

$$\binom{8 - d_x}{2} \leq c + q_m - 1. \quad (8)$$

For $d_x \geq 7$, interpret the binomial term on the left as zero; this is the local-forest lower bound when the sink has at most two A neighbours.

In particular, if d_0 is the smallest nonnegative $d \leq q_M$ satisfying (8), every minority sink has $d_x \geq d_0$. Two distinct majority cores have at most two common cube neighbours. Double-counting pairs of core neighbours across the c sinks yields

$$c \binom{d_0}{2} \leq \sum_{x \in H_m} \binom{d_x}{2} \leq 2 \binom{q_M}{2} = q_M(q_M - 1). \quad (9)$$

1.5 Cases excluded by the current proof

No independent- B or monochromatic-sink counterexample. If B is an independent feedback vertex set, its vertices lie in one cube parity: choose a minimum-distance opposite-parity pair $x, y \in B$. Their interval subcube contains no other B vertex, since every interior vertex is closer to both endpoints and has opposite parity to one of them. Independence makes the odd distance at least three. Removing only the two antipodes from Q_3 leaves a six-cycle; for a

larger odd interval choose a square avoiding both endpoints. Either case contradicts acyclicity of A . Thus B lies in one parity, and the complementary vertices of that parity form a distance-four code (a distance-two pair would leave a square in A). The code bound gives $b \geq 256 - 20 = 236$, contrary to $b \leq 232$.

More generally, if all B -sinks H have one parity, code deletion and an exact finite relaxation exclude every $\delta = 0, \dots, 7$. Here is the certificate, included in full. Assume H is even, write q_O, q_E for odd and even core counts. If $q_O = 0$, then every B vertex is even; there can be no internal B edge, so B is independent and the preceding argument applies. Hence $1 \leq q_O \leq 9$. For each odd core u write a_u, i_u, t_u for its A -neighbour count, incoming B -degree, and outgoing B -degree. The local lemma forces $a_u \leq 5$ and $\binom{a_u-1}{2} \leq q_O - 1$ for $a_u \geq 2$. We have $a_u + i_u + t_u = 9$, $p(u) \geq a_u + 2i_u \geq 2$, $i_u \leq q_E$. The even A -set has $24 + \delta + q_O$ vertices. For each odd core, delete all but one of its even A neighbours. Every surviving pair at distance two would otherwise leave an A -square, so the code bound gives

$$\sum_{u \in C_O} (\max(a_u - 1, 0) - 1) \geq \delta + 4. \quad (10)$$

The zero-neighbour case contributes -1 to this sum. The following dynamic program enumerates a *superset* of possible local types: replacing actual $p(u)$ by the lower bound $a_u + 2i_u$ can only relax the cost and excess restrictions. It keeps the greatest gain at each exact pair (cost, excess), so its state merging is logically safe. Each even core costs at least seven. Every assertion is over integers and uses only Python's standard library.

```
from math import comb

def monochromatic_audit(delta):
    maximum_gap = None
    for q_odd in range(1, 10):
        for q_even in range(10 - q_odd):
            types = []
            for a in range(6):
                if a >= 2 and comb(a - 1, 2) > q_odd - 1:
                    continue
                for incoming in range(q_even + 1):
                    outgoing = 9 - a - incoming
                    p_lower = a + 2 * incoming
                    if not (1 <= outgoing <= 8 and p_lower >= 2):
                        continue
                    types.append(((p_lower - 1) * outgoing,
                                   (p_lower - 2) * outgoing,
                                   max(a - 1, 0) - 1))
            dp = {(0, 0): 0}
            for _ in range(q_odd):
                next_dp = {}
                for (cost, excess), gain in dp.items():
                    for add_cost, add_excess, add_gain in types:
                        key = (cost + add_cost, excess + add_excess)
                        if key[0] + 7 * q_even > 63 or key[1] > 8 * delta:
                            continue
                        next_dp[key] = max(next_dp.get(key, -10**9),
                                           gain + add_gain)
                dp = next_dp
            for gain in dp.values():
                gap = gain - (delta + 4)
                maximum_gap = gap if maximum_gap is None else max(maximum_gap, gap)
                assert gap < 0, (delta, q_odd, q_even, gain)
    return maximum_gap
```

```
assert [monochromatic_audit(delta) for delta in range(8)] == [
    -4, -4, -3, -3, -2, -1, -2, -3]
```

This certificate gives a mathematical exclusion because every actual monochromatic-sink labelling contributes one enumerated type per odd core, meets all three necessary bounds, and would violate the final assertion. It does not certify a mixed-sink case.

At most six cores in a mixed-sink configuration. Under (7), $1 \leq c \leq 12$. Enumerating (8)–(9) for $q_M + q_m \leq 6$ leaves only $(c, q_M, q_m, d_0) = (2, 6, 0, 6)$. Then both minority sinks meet all six majority cores, so they have six common neighbours; two distinct cube vertices have at most two. This is impossible. The tiny full integer check is:

```
from math import comb
survivors = []
for c in range(1, 13):
    for q_M in range(7):
        for q_m in range(7 - q_M):
            d_0 = next((d for d in range(q_M + 1)
                        if comb(8 - d, 2) <= c + q_m - 1), None)
            if d_0 is not None and c * comb(d_0, 2) <= q_M * (q_M - 1):
                survivors.append((c, q_M, q_m, d_0))
assert survivors == [(2, 6, 0, 6)]
```

Nine cores. If $q = 9$, (3)–(4) force $W = 63$, $s = 1$, and $P = 2368$. Each core costs exactly seven, so it is either a *needle* with $(p, t, a, i) = (8, 1, 8, 0)$ or a *hub* with $(p, t, a, i) = (2, 7, 2, 0)$. In particular C is independent and all its outgoing B -edges end in H . If h cores are hubs, then $e_B = 9 + 6h$ and (1) gives $8b = 1802 + 6h$. The only integral possibilities are $(h, b) = (1, 226), (5, 229), (9, 232)$. The corresponding numbers $N = 9 - h$ of needles are 8, 4, 0, with $\delta = 6, 3, 0$.

The monochromatic- H case is excluded above, so take H mixed and its minority part of size c . Equation (7) gives $c \leq 11$ for $N = 8$ and $c \leq 10$ for $N = 4$ or 0: the allowed neighbourhood surpluses are, respectively, $24 + 6 + 9 = 39$, $24 + 3 + 9 = 36$, and $24 + 0 + 9 = 33$. No needle has minority parity. Its eight A neighbours create $\binom{8}{2} - 7 = 21$ required other B vertices of that parity by (6), but at most $c + 9 - 1 \leq 19$ are available. At most one hub has majority parity: two such hubs have seven minority sink neighbours each, and their overlap is at most two, forcing $c \geq 12$.

Let $k \in \{0, 1\}$ count majority hubs. Then the majority and minority core counts are $m = N + k$ and $n = 9 - m$. The total incidence count from majority cores to minority sinks is $D = N + 7k$. Every minority sink x has a core-neighbour count $d_x \leq m$, with $\sum_x d_x = D$ and $\binom{8-d_x}{2} \leq c + n - 1$ by (8). Two minority sinks share at most two such neighbours. These inequalities finish each numerical case:

- If $N = 0$, then $m = k \leq 1$, $c \leq 10$, and every minority sink has at least eight A neighbours. It requires at least 21 other minority-parity B vertices by (6), but there are at most $c + n - 1 \leq 18$.
- If $N = 4, k = 0$, then $(m, n, D) = (4, 5, 4)$ and $c \leq 10$. Equation (8) forces every $d_x \geq 3$, hence $c = 1$. At $c = 1$ it forces $d_x \geq 5 > m$.
- If $N = 4, k = 1$, then $(m, n, D) = (5, 4, 11)$ and $c \leq 10$. First (8) forces $d_x \geq 3$ and $c \leq 3$; then it forces $d_x \geq 4$ and $c \leq 2$. But $D = 11$ and $d_x \leq 5$ demand $c \geq 3$.
- If $N = 8, k = 1$, then $(m, n, D) = (9, 0, 15)$ and $c \leq 11$. Applying (8) successively gives $d_x \geq 3, c \leq 5$; then $d_x \geq 5, c \leq 3$; then $d_x \geq 6, c \leq 2$. There cannot be one sink because $D = 15 > m$. Two sink neighbour sets of total size 15 in a nine-element core set overlap in at least six vertices, but two cube vertices share at most two neighbours.

- If $N = 8, k = 0$, then $(m, n, D) = (8, 1, 8)$ and $c \leq 11$. Equation (8) gives $d_x \geq 3, c \leq 2$, then $d_x \geq 6, c = 1, d_x = 8$. The sole minority sink has one A neighbour; the sole minority core is a hub with two. The majority A -set has $256 - (226 - 2) = 32$ vertices. Deleting all but one A neighbour for each of the two minority B vertices removes at most one vertex and leaves a distance-four code of at least 31 words, contradicting $A(9, 4) \leq 20$.

Thus $q = 9$ is impossible, including the $\delta = 0$ branch. This uses only the assumed $P \leq 2368$, not the organizer's unpublished lower-bound proof.

One minority sink and at most eight cores. Assume $0 \leq \delta \leq 7, q \leq 8$, and $c = 1$. Let x be the minority sink, n the minority core count, m the majority core count, and $d \leq m$ the number of core neighbours of x . Equation (8) becomes $\binom{8-d}{2} \leq n$, while $d + n \leq 8$. The only possibilities are

$$(d, n) = (5, 3), (6, 1), (6, 2), (7, 0), (7, 1), (8, 0).$$

For each minority core u , (6) gives $\binom{a_u-1}{2} \leq n$. Thus $a_u \leq 3$ for $n \leq 2$, and $a_u \leq 4$ for $n = 3$. Apply the code-deletion argument to the majority-parity A -set. With $g(a) = \max(a - 1, 0) - 1$, it requires $g(9 - d) + \sum g(a_u) \geq \delta + 4$. The sink gain is $g(9 - d) = 7 - d$, including the case $d = 8$, when it has one A neighbour. For $n \leq 2$, total gain is at most three, whereas $\delta + 4 \geq 4$. Only $(d, n) = (5, 3)$ remains; then $m = 5, q = 8$, the sink gain is two and the other three gains are at most two each, so $\delta \leq 4$.

Every majority core can have outgoing B -edges only to those three minority cores and x , so $1 \leq t \leq 4$. By $p \geq 9 - t$, each contributes $(p - 2)t \geq (7 - t)t \geq 6$ to E . The five contribute at least 30, forcing $\delta \geq 4$. Equality in the gain bound at $\delta = 4$ requires $a_u = 4$ for all three minority cores. If such a core has incoming B -degree $i \in \{0, \dots, 4\}$, then $t = 5 - i, p \geq 4 + 2i$, and its excess is at least $(2 + 2i)(5 - i) \geq 10$. The three minority cores add at least 30 more, giving $E \geq 60 > 8\delta = 32$, a contradiction.

Two minority sinks. Suppose $c = 2$ and put $n = q_m, m = q_M$. Each sink needs at least d majority-core neighbours, where d is the least integer satisfying $\binom{8-d}{2} \leq n + 1$. Their two neighbour sets overlap in at most two cores, so $m \geq 2d - 2$ and $q = m + n \geq n + 2d - 2$. For $n = 0, \dots, 9$ the latter lower bounds are, in order, 10, 11, 10, 11, 12, 11, 12, 13, 14, 13. Thus $q \geq 10$, contradicting $q \leq 9$. This argument covers $c = 2$ even when $q = 9$; the nine-core proof above is an independent cross-check.

At least three minority sinks and seven or eight cores. The remaining arithmetic case has $q \in \{7, 8\}, 3 \leq c \leq 11$, and $\delta \in \{0, \dots, 7\}$. The sharper upper bound $c \leq 11$ follows from (7): here $24 + \delta + q \leq 39$, whereas Harper's surplus is 40 at $c = 12$ and at least 42 for $13 \leq c \leq 116$. Write $m = q_M$ and $n = q_m$. The case $m = 0$ is already impossible: every minority sink then has $d_x = 0$, so (8) would require $\binom{8}{2} = 28 \leq c + n - 1 \leq 11 + 8 - 1 = 18$. Thus the inventory below may take $1 \leq m \leq q$.

For each majority core y_j , let e_j be its number of minority-sink neighbours, i_j its incoming B -degree, and t_j its outgoing B -degree. Since sinks have no outgoing edges, incoming B neighbours lie in minority C . Therefore

$$0 \leq i_j \leq n, \quad e_j \leq t_j \leq e_j + n - i_j, \quad 1 \leq t_j \leq 8, \quad a_j = 9 - i_j - t_j \geq 0, \quad p(y_j) \geq a_j + 2i_j \geq 2. \quad (11)$$

The finite checker below enumerates all such local types. It uses the lower cost $w_j = (a_j + 2i_j - 1)t_j$ and lower excess $z_j = (a_j + 2i_j - 2)t_j$. Replacing actual $p(y_j)$ by its lower bound can only make a hypothetical labelling easier to satisfy.

Sort the m values e_j and put $D = \sum_j e_j = \sum_{x \in H_m} d_x$. Sorting loses no configuration because

all tests are invariant under permutation. Let $d_{\min}$ be the least $d \leq m$ satisfying (8). Then $D \geq cd_{\min}$. A pair of same-parity cube vertices has at most two common neighbours. Applied to both sides of this incidence graph, this gives

$$\sum_j \binom{e_j}{2} \leq c(c-1), \quad \sum_x \binom{d_x}{2} \leq m(m-1), \quad e_j + e_k \leq c+2. \quad (12)$$

For fixed D , the middle left side is least when the c values d_x differ by at most one. If $D = cu + r$, $0 \leq r < c$, its minimum is $(c-r)\binom{u}{2} + r\binom{u+1}{2}$. Using this minimum is a relaxation, so it cannot discard a real graph.

The A vertices in the majority parity number $24 + \delta + n + c$. Delete all but one of their neighbours for each minority B vertex. The survivors form a distance-four code, as before. The c minority sinks contribute deletion gain $\sum_x (7 - d_x) = 7c - D$. For a minority core with a A neighbours, (6) requires $\binom{a-1}{2} \leq c + n - 1$; let $a_{\max}$ be the greatest allowed a . The code bound therefore requires

$$7c - D + n(\max(a_{\max} - 1, 0) - 1) \geq 4 + \delta. \quad (13)$$

Independent minima of the local types give the further necessary inequalities

$$\sum_j \min w_j + 7n \leq 63, \quad \sum_j \min z_j \leq 8\delta. \quad (14)$$

The minima need not be simultaneously attainable; taking them separately only enlarges the candidate set.

Two final inequalities make the finite relaxation empty. First, the neighbour sets of any three majority cores are subsets of the c minority sinks, with each pairwise intersection of size at most two. Inclusion–exclusion gives

$$e_j + e_k + e_\ell \leq c + 6. \quad (15)$$

It is enough to test the three largest values.

Second, a core of incoming B -degree i has cost $(p-1)t \geq 7+i$. For $i=0$ this is (4). For $i \geq 1$, $p \geq 9-t+i$ gives

$$(p-1)t - (7+i) \geq (t-1)(7+i-t) \geq 0$$

because $1 \leq t \leq 8$. Let J count edges directed from majority cores to minority cores. The minority cores cost at least $7n + J$. A majority core with local type (e, i, t) sends exactly $t - e$ edges to minority cores; the other outgoing edges go to minority sinks. Thus every labelling must satisfy the *strengthened* bound

$$7n + \sum_j \min_{\text{allowed local types}} [w_j + (t_j - e_j)] \leq 63. \quad (16)$$

Again, each minimum is taken independently, which is safe for exclusion.

The following standard-library-only checker is the full finite inventory for $\delta = 0, \dots, 7$, $q = 7, 8$, all $m + n = q$, all $3 \leq c \leq 11$, and every nondecreasing m -tuple $(e_1, \dots, e_m)$ with entries from 0 to c . It implements (7)–(8) and (11)–(16), checks the core cost lemma over every local type, and asserts that no profile survives. There is no SAT/SMT solver, timeout, sampled search, or assumption that a degree profile is realizable. The reported counts are 1,110,562 enumerated multisets, 134 after the coarse necessary filters, 20 after (15), and zero after (16). The 20 rows have strengthened costs 68, 71, or 72, all greater than 63.

```
#!/usr/bin/env python3
"""Independent full arithmetic replay for mixed H, q=7/8, c=3..11.
```

All filters are necessary consequences of the stated graph/path lemmas. The final filter uses the incoming-core edge cost debt. No solver or graph search.

```

"""
from __future__ import annotations
from collections import Counter
from itertools import combinations, combinations_with_replacement
from math import comb
import json

def bin2(k: int) -> int:
    return k*(k-1)//2

def harper_table() -> list[int]:
    covered=set()
    out=[0]
    for weight in range(9):
        for bits in combinations(range(8),weight):
            v=sum(1<<bit for bit in bits)
            covered.add(v)
            covered.update(v^(1<<bit) for bit in range(8))
            out.append(len(covered))
    return out

def core_options(e: int, minor_cores: int):
    choices=[]
    for incoming in range(minor_cores+1):
        for outgoing in range(1,9):
            a=9-incoming-outgoing
            if a<0:continue
            p_lower=a+2*incoming
            if p_lower<2:continue
            outgoing_other_cores=outgoing-e
            if not (0<=outgoing_other_cores<=minor_cores-incoming):continue
            choices.append({
                'a':a,'i':incoming,'t':outgoing,'e':e,
                'cost':(p_lower-1)*outgoing,
                'excess':(p_lower-2)*outgoing,
                'cost_with_debt':(p_lower-1)*outgoing+outgoing_other_cores,
            })
    return choices

def valid_row_summaries():
    harp=harper_table()
    counts=Counter();after_triples=[];before_triples=[]
    for delta in range(8):
        for q in (7,8):
            for minor_cores in range(q):
                major_cores=q-minor_cores
                for c in range(3,12):
                    if harp[c]-c>24+delta+q:continue
                    maximum_a=max(a for a in range(10)
                                if a<2 or bin2(a-1)<=c+minor_cores-1)
                    minimum_d=next((d for d in range(major_cores+1)
                                if bin2(8-d)<=c+minor_cores-1),None)
                    if minimum_d is None:continue
                    if c*bin2(minimum_d)>major_cores*(major_cores-1):continue
                    options={e:core_options(e,minor_cores) for e in range(c+1)}
                    for degrees in combinations_with_replacement(range(c+1),major_cores):

```

```

counts['degree_multisets']+=1
D=sum(degrees)
if D<c*minimum_d:continue
small,rem=divmod(D,c)
if (c-rem)*bin2(small)+rem*bin2(small+1)>major_cores*(major_cores-1):continue
if sum(bin2(e) for e in degrees)>c*(c-1):continue
if any(x+y>c+2 for x,y in combinations(degrees,2)):continue
gain=7*c-D+minor_cores*(max(maximum_a-1,0)-1)
if gain<4+delta:continue
if any(not options[e] for e in degrees):continue
lower_cost=sum(min(t['cost'] for t in options[e]) for e in
    ↪ degrees)+7*minor_cores
lower_excess=sum(min(t['excess'] for t in options[e]) for e in degrees)
if lower_cost>63 or lower_excess>8*delta:continue
row={'delta':delta,'q':q,'c':c,'major_cores':major_cores,
    'minor_cores':minor_cores,'minimum_d':minimum_d,
    'degrees':degrees,'lower_cost':lower_cost,
    'lower_excess':lower_excess}
before_triples.append(row)
counts['before_triple']+=1
if any(sum(triple)>c+6 for triple in combinations(degrees,3)):
    continue
counts['after_triple']+=1
strengthened=sum(min(t['cost_with_debt'] for t in options[e])
    for e in degrees)+7*minor_cores
row['strengthened_cost']=strengthened
after_triples.append(row)
if strengthened<=63:counts['after_strengthened_cost']+=1
assert counts['after_strengthened_cost']==0
return counts,before_triples,after_triples

def main():
    # Directly audit the helper lemma for all local types.
    for a in range(10):
        for incoming in range(10-a):
            outgoing=9-a-incoming
            if not (1<=outgoing<=8 and a+2*incoming>=2):continue
            assert (a+2*incoming-1)*outgoing>=7+incoming
    counts,before,after=valid_row_summaries()
    print(json.dumps({'counts':counts,'after_triple_rows':after,
        'before_triple_count':len(before)},indent=2,sort_keys=True))

if __name__=='__main__':main()

```

Every filter is a necessary condition for a genuine labelling. The nested loops cover the entire stated integer range, including $\delta = 0$, and the final zero assertion therefore excludes every remaining $q = 7, 8$ mixed-sink case with $c \geq 3$.

1.6 Conclusion and verification status

Assume $P \leq 2368$. The exact budget gives $225 \leq b \leq 232$ and $q \leq 9$. If H is monochromatic, the code-deletion certificate excludes it. If H is mixed, Harper gives $c \leq 12$; the cases $q \leq 6$, $c = 1$ with $q \leq 8$, $c = 2$, $q = 9$, and $3 \leq c \leq 11$ with $q = 7, 8$ are excluded above. These cases exhaust all possibilities, contradicting $P \leq 2368$. Therefore $U(Q_9) \geq 2369$, improving the stated 2368 lower bound.

1.7 Replaying the finite certificates

The four Python blocks above are independent standard-library programs: the Harper initial-segment table, the monochromatic-sink dynamic program, the at-most-six-core check, and the final mixed-degree inventory. Save this Markdown as `cagent/p2/c5/submission.md` at the repository root and run:

```
python3 - <<'PY'
from pathlib import Path
import re
import subprocess
import sys
from time import perf_counter

text = Path("cagent/p2/c5/submission.md").read_text(encoding="utf-8")
blocks = re.findall(r"```python\n(.*?)```", text, re.S)
assert len(blocks) == 4, len(blocks)
start = perf_counter()
for number, block in enumerate(blocks, 1):
    result = subprocess.run([sys.executable, "-c", block],
                           check=True, capture_output=True, text=True)
    print(f"block {number}: PASS")
    if number == 4:
        print(result.stdout)
print(f"elapsed_seconds={perf_counter() - start:.6f}")
PY
```

The four blocks completed in under one second in an independently recorded Python 3.14.5 replay. The final block enumerated 1,110,562 degree multisets and reported 134 after the coarse restrictions, 20 after the triple-union restriction, and zero after the strengthened cost restriction.